

Instructor: Date:

Learner: Regional Branch:

Program: Center:

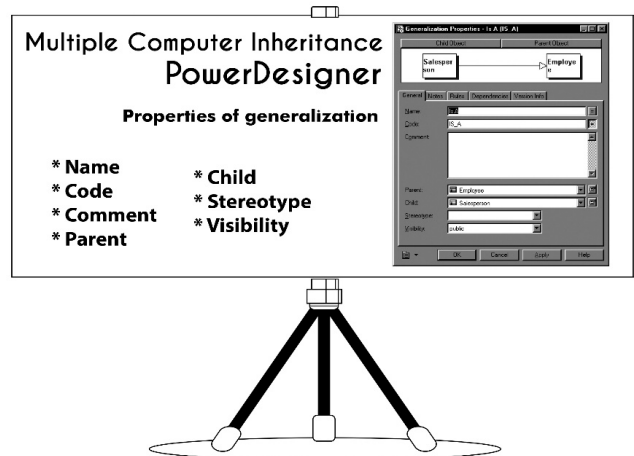
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Talking About Computer Programming



Learning Outcome:

Describe common activities at work to a visitor using short sentences in familiar spoken and written situations.
Talk about particular topics related to software programming.

Can Do Statements:

- I can explain the concept of inheritance.
- I can give detailed information about inheritance.
- I can talk about the relation between object-oriented programming and inheritance.

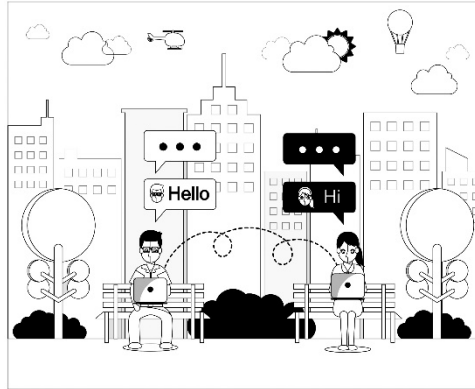
Ask your partner:

- What is computer programming?
- How do you think that computer programming works?



Let's Explore!

Studying for the Test About Inheritance



Instructions: Read the text below.

- Tanya:** Mark, we got to study for our test. It's about inheritance.
- Mark:** I used to think that the term inheritance did not apply to programming languages.
- Tanya:** It does! That's why the teacher is including it in the quiz. Let's clarify what inheritance is.
- Mark:** Inheritance is basing an object or class on another object by using the same one or by identifying a new implementation to maintain the same behavior. This inherited class is called a subclass.
- Tanya:** To relate objects or classes through inheritance creates hierarchy. I mean, inheritance builds a syntactic relationship among objects and classes.
- Mark:** I used to believe that inheritance created a semantic relationship.
- Tanya:** Not necessarily. It doesn't even ensure a behavioral subtyping.
- Mark:** What about the types of inheritance? I know that people used to classify inheritance based on specific languages.
- Tanya:** That's right! There is the single inheritance, where the class gets the properties of another class; or the multiple inheritance, where one class inherits characteristics from all parent classes.
- Mark:** I used to know about hierarchical inheritance, but I don't remember well.
- Tanya:** It's when one class assists as a superclass. Another important type of inheritance is the hybrid one. Mixing two or more types of inheritance is called hybrid inheritance.
- Mark:** But there is something we need to know, what's the relation between this topic and object-oriented programming? Is it the same?
- Tanya:** Well, object-oriented programming is a programming paradigm that involves objects that have data as fields, known as attributes; and codes which we call methods. Inheritance is just a feature of this paradigm.
- Mark:** It also uses objects, right?
- Tanya:** Of course! To design computer programs, it requires objects that interact with one another.
- Mark:** I feel ready for that test now!



Let's Practice!

Inheritance for everybody

Instructions: Circle the correct option to complete the sentences.

1. (To inherit / To inheriting) commonly used classes from other objects is a feature from Object-oriented programming.
2. Object-oriented programming also inherits behavior from other classes by (to use / using) its characteristics.
3. (Inherit / Inheriting) the behavior from another class makes the first class become a superclass.



Let's Practice!

Extra inheritance exercise

Instructions: Complete the sentences by writing its corresponding word. Use capital letter where necessary. Use: to say, ensure, think.

1. I used to _____ that a defined subclass couldn't inherit the definitions of one or more general classes, I was wrong.
2. We used _____ that inheritance makes program development slower, but in fact, it makes it faster.
3. They used to _____ the validity of the defined subclass object through inheritance.





Extension Activity

Inheritance vs Subtyping

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Investigate about the difference between inheritance and subtyping.
- Write your findings about the difference between inheritance and subtyping in your notebook.
- Share your findings on the difference between inheritance and subtyping with your classmates and teacher.

Self-Assessment

Yes No

- I investigate about the difference between inheritance and subtyping.
- I write about the difference between inheritance and subtyping.
- I talk about the difference between inheritance and subtyping.

☐	☐
☐	☐
☐	☐



Target Words

- **subclass** (noun): A subdivision of a set of classes
e.g. This inherited class is called a subclass.
- **implementation** (noun): the act of putting something into practice
e.g. Inheritance is when an object or class is based on another object by using the same one or by identifying a new implementation.
- **behavioral** (adjective): how something acts
e.g. It doesn't even ensure a behavioral subtyping.
- **single** (adjective): something that is not related to others
e.g. There is single inheritance where a class gets the properties of another class.
- **assist** (noun): to contribute or support another process
e.g. It's when one class assists as a superclass.
- **feature** (noun): a characteristic of something or somebody
e.g. Codes which are called methods. Inheritance is just a feature of this paradigm.



Learning Points

Used to

Used to expresses an idea of a past habit.

The expression **used to** introduces the idea that someone had a habit, a recurrent action in the past but that it is no longer happening in the present. It can also be used to describe the reality in the past, without making reference to a specific action of a person.

The structure for creating sentences with this expression is the following:

Subject + **used to** + verb (base form) + complement.

For example:

We **used to** use this software programming system 5 years ago. (regular action in the past)

That system **used to** be the best one available in the market. (description of the past reality)

Examples

- *I used to think that the term inheritance was not applied in language programming.*
- *I know that people used to classify inheritance based on specific language.*
- *I used to know about hierarchical inheritance.*

Gerunds vs infinitives

A verb in **gerund** is a verb in its "ing" form. An **infinitive** verb is a verb formed with the preposition "to" plus the base form of the verb.

There are different uses for gerund verbs and infinitives:

GERUNDS:

- They can appear at the beginning of a sentence, used as a subject.

Example:

Designing new programs can be expensive.

- They can be the object following the verb.

Example:

He quitted **working** in this company.



Learning Points

- They can be used after certain verbs (love, hate, enjoy, etc...)

Example:

*I love **making** new projects.*

- After prepositions.

Example:

*I'm good at **working** with objects.*

INFINITIVES:

- Us them as objects following the verb.

Example:

*I forgot **to call** the new software programmer.*

- They are also used as a subject at the beginning of a sentence.

Example:

***To create** a new software is crucial for the company.*

- Some verbs are followed by a noun or pronoun and then by an infinitive.

Example:

*He advised me **to sell** the software.*

Examples

- *To relate objects or classes through inheritance creates hierarchy.*
- *Mixing two or more types of inheritance is called hybrid inheritance.*



Learning Points

What is a compound noun?

A **compound noun** is a noun that is formed with two or more words.

A compound noun is usually formed with these combinations:

- Noun + noun (football)
- Adjective + noun (software)
- Verb + noun (swimming pool)
- Verb + preposition (check-out)
- Noun + prepositional phrase (mother-in-law)
- Preposition + noun (underworld)

A **compound noun** acts as a single unit and can be modified by adjectives and other nouns.

There are three forms for compound nouns:

1. Open or spaced - space between words
2. Hyphenated - hyphen between words
3. Closed or solid - no space or hyphen between words

